

Estimativa estocástica de π com OpenMP

Eugênio Vitor Lopes dos Santos

DCA3703 – Programação Paralela, Universidade Federal do Rio Grande do Norte

1 Enunciado

Implementar em C uma estimativa de π pelo método de Monte Carlo. O programa deve sortear pontos em um quadrado de lado 2 e contar os que pertencem ao círculo unitário. A atividade pede a paralelização do laço principal com OpenMP, a identificação de uma condição de corrida no contador e duas formas de correção: uma seção `critical` a cada incremento e um acumulador local por thread. Também são exploradas as cláusulas `shared`, `private`, `firstprivate`, `lastprivate` e `default(none)`.

2 Fundamentação teórica

2.1 Método de Monte Carlo

O quadrado tem área 4 e o círculo unitário tem área π . Se os pontos forem uniformes no quadrado, a fração de pontos dentro do círculo se aproxima da razão entre as áreas:

$$\frac{n_{\text{dentro}}}{N} \approx \frac{\pi}{4}.$$

A estimativa calculada pelo programa é, portanto,

$$\hat{\pi} = 4 \cdot \frac{n_{\text{dentro}}}{N}.$$

Cada ponto (x, y) pertence ao círculo quando $x^2 + y^2 \leq 1$. Como o método usa amostras aleatórias, o resultado é uma aproximação. Aumentar N reduz a variação, mas não elimina o caráter estocástico da estimativa.

2.2 Condição de corrida

A expressão `dentro_circulo++` lê o contador, soma 1 e grava o novo valor. Essas operações não formam uma única instrução atômica. Duas threads podem ler o mesmo valor antes que qualquer uma grave o resultado. Nesse caso, uma escrita substitui a outra e um incremento desaparece. O programa termina normalmente, mas a contagem fica menor do que deveria.

2.3 Escopo das variáveis em OpenMP

Dentro de uma região paralela, `shared` mantém uma única variável para todo o time de threads. `private` cria uma cópia sem valor inicial definido para cada thread. `firstprivate` também cria cópias, mas inicia cada uma com o valor que a variável tinha antes da região paralela. Em um laço, `lastprivate` copia para a variável externa o valor produzido na última iteração lógica, e não o valor da thread que terminou por último.

A cláusula `default(none)` obriga a declarar o escopo das variáveis referenciadas na região paralela. O compilador passa a rejeitar uma variável que tenha sido esquecida, em vez de assumir um escopo implícito.

3 Implementação

A versão sequencial, `e0.c`, usa uma semente única e atualizada por `rand_r`. Ela fornece a referência para o cálculo. Em `e1.c`, cada índice do laço inicializa uma semente determinística e independente. Assim, as execuções recebem os mesmos pontos e a variação observada vem da condição de corrida no contador compartilhado.

Em `e2.c`, a diretiva `critical` protege somente o incremento de `dentro_circulo`. O sorteio e o teste geométrico ainda ocorrem em paralelo, mas todas as threads disputam a mesma seção crítica milhões de vezes.

A versão `e3.c` separa as diretivas `parallel` e `for`. Cada thread declara `local_dentro` e uma semente própria antes do laço. Durante o `for`, ela atualiza apenas o contador local. Depois do laço, cada thread entra uma vez em `critical` para somar seu resultado no contador global. A diretiva também usa `default(none)` e declara `total_elementos` e `dentro_circulo` como `shared`.

Para validar isoladamente as cláusulas de compartilhamento e escopo de dados em OpenMP, foram desenvolvidos quatro programas complementares:

- `e4_private.c`: demonstra a criação de cópias locais sem inicialização;
- `e4_firstprivate.c`: demonstra a inicialização da cópia local com o valor prévio;
- `e4_lastprivate.c`: demonstra a propagação do valor da última iteração lógica;
- `e4_default_none.c`: demonstra a obrigatoriedade de classificação explícita.

```
1 #define _DEFAULT_SOURCE
2
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <time.h>
6 #include <math.h>
7
8 #define TOTAL_ELEMENTOS_PADRAO 5000000L
9
10 int main(int argc, char *argv[]) {
11     long long total_elementos = argc > 1 ? atoll(argv[1]) :
        TOTAL_ELEMENTOS_PADRAO;
12
13     long long dentro_circulo = 0;
14
15     unsigned int seed = time(NULL);
16
17     for (long long i = 0; i < total_elementos; i++) {
```

```

18     double x = 2.0 * (double)rand_r(&seed) / (double)RAND_MAX - 1.0;
19     double y = 2.0 * (double)rand_r(&seed) / (double)RAND_MAX - 1.0;
20
21     if ((x * x + y * y) <= 1.0) {
22         dentro_circulo++;
23     }
24 }
25
26 double pi = 4.0 * dentro_circulo / total_elementos;
27 double erro = fabs(pi - M_PI);
28
29 printf("Dentro do circulo = %lld\nPI = %.10f\nErro = %.10e\n",
30        dentro_circulo, pi, erro);
31
32 return 0;
33 }

```

Listing 1: e0.c, versão sequencial

```

1 #define _DEFAULT_SOURCE
2
3 /*
4  * dentro_circulo++ envolve ler, somar 1 e escrever de volta.
5  * Sao pelo menos 3 instrucoes de maquina que podem se entrelacar
6  * entre threads, perdendo incrementos.
7  */
8
9 #include <stdio.h>
10 #include <stdlib.h>
11 #include <math.h>
12 #include <omp.h>
13
14 #define TOTAL_ELEMENTOS_PADRAO 5000000L
15
16 int main(int argc, char *argv[]) {
17     long long total_elementos = argc > 1 ? atoll(argv[1]) :
18         TOTAL_ELEMENTOS_PADRAO;
19
20     long long dentro_circulo = 0;
21
22     #pragma omp parallel for
23     for (long long i = 0; i < total_elementos; i++) {
24         unsigned int seed = (unsigned int)(i + 1);
25         double x = 2.0 * (double)rand_r(&seed) / (double)RAND_MAX - 1.0;
26         double y = 2.0 * (double)rand_r(&seed) / (double)RAND_MAX - 1.0;
27
28         if ((x * x + y * y) <= 1.0) {
29             dentro_circulo++;
30         }
31     }
32 }

```

```

31
32     double pi = 4.0 * dentro_circulo / total_elementos;
33     double erro = fabs(pi - M_PI);
34
35     printf("Dentro do circulo = %lld\nPI = %.10f\nErro = %.10e\n",
36           dentro_circulo, pi, erro);
37
38     return 0;
39 }

```

Listing 2: e1.c, versão com condição de corrida

```

1  #define _GNU_SOURCE
2  #define _POSIX_C_SOURCE 199309L
3
4  #include <stdio.h>
5  #include <stdlib.h>
6  #include <math.h>
7  #include <omp.h>
8  #include <time.h>
9
10 #define TOTAL_ELEMENTOS_PADRAO 5000000L
11
12 static double obter_tempo_segundos(void) {
13     struct timespec tempo_atual;
14     clock_gettime(CLOCK_MONOTONIC, &tempo_atual);
15     return (double)tempo_atual.tv_sec + (double)tempo_atual.tv_nsec / 1e9;
16 }
17
18 int main(int argc, char *argv[]) {
19     long long total_elementos = argc > 1 ? atoll(argv[1]) :
20         TOTAL_ELEMENTOS_PADRAO;
21
22     long long dentro_circulo = 0;
23
24     double tempo = obter_tempo_segundos();
25
26     #pragma omp parallel for
27     for (long long i = 0; i < total_elementos; i++) {
28         unsigned int seed = (unsigned int)(i + 1);
29         double x = 2.0 * (double)rand_r(&seed) / (double)RAND_MAX - 1.0;
30         double y = 2.0 * (double)rand_r(&seed) / (double)RAND_MAX - 1.0;
31
32         if ((x * x + y * y) <= 1.0) {
33             #pragma omp critical
34             {
35                 dentro_circulo++;
36             }
37         }
38     }
39 }

```

```

38
39     double pi = 4.0 * dentro_circulo / total_elementos;
40     double erro = fabs(pi - M_PI);
41
42     tempo = obter_tempo_segundos() - tempo;
43
44     printf("PI = %.10f\n", pi);
45     printf("Erro = %.10e\n", erro);
46     printf("Tempo = %.9f segundos\n", tempo);
47
48     return 0;
49 }

```

Listing 3: e2.c, incremento protegido por critical

```

1  #define _GNU_SOURCE
2  #define _POSIX_C_SOURCE 199309L
3
4  #include <stdio.h>
5  #include <stdlib.h>
6  #include <math.h>
7  #include <omp.h>
8  #include <time.h>
9
10 #define TOTAL_ELEMENTOS_PADRAO 5000000L
11
12 static double obter_tempo_segundos(void) {
13     struct timespec tempo_atual;
14     clock_gettime(CLOCK_MONOTONIC, &tempo_atual);
15     return (double)tempo_atual.tv_sec + (double)tempo_atual.tv_nsec / 1e9;
16 }
17
18 int main(int argc, char *argv[]) {
19     long long total_elementos = argc > 1 ? atoll(argv[1]) :
        TOTAL_ELEMENTOS_PADRAO;
20
21     long long dentro_circulo = 0;
22
23     double tempo = obter_tempo_segundos();
24
25     #pragma omp parallel default(none) shared(total_elementos,
        dentro_circulo)
26     {
27         long long local_dentro = 0;
28         unsigned int seed = 123456789U ^ (unsigned int)omp_get_thread_num
            ();
29
30         #pragma omp for
31         for (long long i = 0; i < total_elementos; i++) {

```

```

32     double x = 2.0 * (double)rand_r(&seed) / (double)RAND_MAX -
        1.0;
33     double y = 2.0 * (double)rand_r(&seed) / (double)RAND_MAX -
        1.0;
34
35     if ((x * x + y * y) <= 1.0) {
36         local_dentro++;
37     }
38 }
39
40 #pragma omp critical
41 {
42     dentro_circulo += local_dentro;
43 }
44 }
45
46 double pi = 4.0 * dentro_circulo / total_elementos;
47 double erro = fabs(pi - M_PI);
48
49 tempo = obter_tempo_segundos() - tempo;
50
51 printf("PI = %.10f\n", pi);
52 printf("Erro = %.10e\n", erro);
53 printf("Tempo = %.9f segundos\n", tempo);
54
55 return 0;
56 }

```

Listing 4: e3.c, contador local por thread

```

1  // e4_private.c - Demonstracao de private(x)
2  // private cria uma copia nao inicializada para cada thread.
3  #include <omp.h>
4  #include <stdio.h>
5
6  int main() {
7      int x = 100;
8
9      #pragma omp parallel for private(x)
10     for (int i = 0; i < 8; i++) {
11         printf("Thread %d: x (antes) = %d\n", omp_get_thread_num(), x);
12         x = omp_get_thread_num();
13         printf("Thread %d: x (depois) = %d\n", omp_get_thread_num(), x);
14     }
15
16     printf("Fora da regioao paralela: x = %d\n", x);
17     return 0;
18 }

```

Listing 5: e4_private.c, demonstração da cláusula private

```

1 // e4_firstprivate.c - Demonstracao de firstprivate(x)
2 // firstprivate cria uma copia E inicializa com o valor original.
3 // Diferente de private: o valor de x=100 esta disponivel dentro do laço.
4 #include <omp.h>
5 #include <stdio.h>
6
7 int main() {
8     int x = 100;
9
10    #pragma omp parallel for firstprivate(x)
11    for (int i = 0; i < 8; i++) {
12        printf("Thread %d: x (antes) = %d\n", omp_get_thread_num(), x);
13        x = omp_get_thread_num();
14        printf("Thread %d: x (depois) = %d\n", omp_get_thread_num(), x);
15    }
16
17    printf("Fora da regioao paralela: x = %d\n", x);
18    return 0;
19 }

```

Listing 6: e4_firstprivate.c, demonstração da cláusula firstprivate

```

1 // e4_lastprivate.c - Demonstracao de lastprivate
2 // lastprivate atribui a variavel o valor da ULTIMA iteracao (i = N-1).
3 // Resultado e sempre i*i = 19*19 = 361, independente da thread.
4 #include <omp.h>
5 #include <stdio.h>
6
7 int main() {
8     int resultado = 0;
9
10    #pragma omp parallel for lastprivate(resultado)
11    for (int i = 0; i < 20; i++) {
12        resultado = i * i;
13        printf("Thread %d: i = %d, resultado = %d\n", omp_get_thread_num(),
14              , i, resultado);
15    }
16
17    printf("Apos o laço: resultado = %d (esperado: %d)\n", resultado, 19 *
18          19);
19    return 0;
20 }

```

Listing 7: e4_lastprivate.c, demonstração da cláusula lastprivate

```

1 // e4_default_none.c - Demonstracao de default(none) com erro proposital
2 // Usar default(none) sem classificar todas as variaveis causa erro de
3 // compilacao.
4 // Isso e proposital: mostra como default(none) pega erros antes do
5 // programa rodar.

```

```

4 #include <omp.h>
5 #include <stdio.h>
6
7 int main() {
8     int x = 10;
9     int y = 20;
10    int soma = 0;
11
12    #pragma omp parallel for default(none) shared(x, soma)
13    for (int i = 0; i < 10; i++) {
14        soma += x + y; // 'y' nao esta classificada!
15    }
16
17    printf("soma = %d\n", soma);
18    return 0;
19 }

```

Listing 8: e4_default_none.c, demonstração de default(none)

Os códigos-fonte são compilados com o GCC utilizando o padrão C11, nível de otimização -O2, avisos ativados (-Wall -Wextra) e suporte a OpenMP (-fopenmp). Para as medições de desempenho, a afinidade das threads é controlada por variáveis de ambiente (OMP_PROC_BIND=close e OMP_PLACES=cores), variando a contagem de threads de 1 até o limite da máquina com dez repetições por configuração.

```

1 gcc -std=c11 -O2 -Wall -Wextra -fopenmp e0.c -o e0 -lm
2
3 OMP_NUM_THREADS=1 ./e0 5000000
4 OMP_NUM_THREADS=28 ./e3 5000000

```

Listing 9: Exemplo de compilação e execução

As flags de compilação têm as seguintes funções: -std=c11 fixa o padrão C11 da linguagem; -O2 aplica otimizações do compilador; -Wall -Wextra ativam diagnósticos adicionais; -fopenmp habilita a interpretação de diretivas OpenMP e a vinculação com a libgomp; e -lm vincula a biblioteca matemática padrão.

4 Resultados e discussão

Os testes usaram $N = 5.000.000$, dez repetições por configuração e um Intel Core i7-14700HX exposto ao WSL2 com 14 núcleos físicos e 28 threads lógicas. A afinidade foi configurada com `OMP_PROC_BIND` close e `OMP_PLACES` cores, e `OMP_DYNAMIC` ficou desativado. A máquina é um laptop sob um hipervisor; o sistema operacional interfere nas medições e o desvio padrão é alto em várias configurações. As tabelas usam a mediana dos tempos; as barras de erro dos gráficos mostram o desvio padrão das dez repetições.

4.1 Versão sem sincronização

Com uma thread, e1.c retornou $\hat{\pi} = 3,1415096$, igual ao resultado de e2.c. Os dois programas usam a mesma sequência de pontos, portanto a diferença a partir de duas threads só pode vir do acesso concorrente ao contador. A Tabela 1 resume as dez execuções de e1.c. A média com 28 threads foi 0,4478194, com valores entre 0,3476832 e 0,5515736.

Table 1: Estimativa de e1.c em dez repetições por configuração.

Threads	Média de $\hat{\pi}$	Desvio padrão	Intervalo observado
1	3,1415096	0,0000000	[3,1415096; 3,1415096]
2	2,0640822	0,1508933	[1,8089600; 2,2215192]
4	0,9132916	0,0891818	[0,7157944; 1,0165576]
8	0,4566514	0,0802879	[0,3161328; 0,5506096]
14	0,4859702	0,1299359	[0,3248536; 0,6791216]
16	0,4114073	0,1164830	[0,2833832; 0,6941400]
28	0,4478194	0,0733959	[0,3476832; 0,5515736]

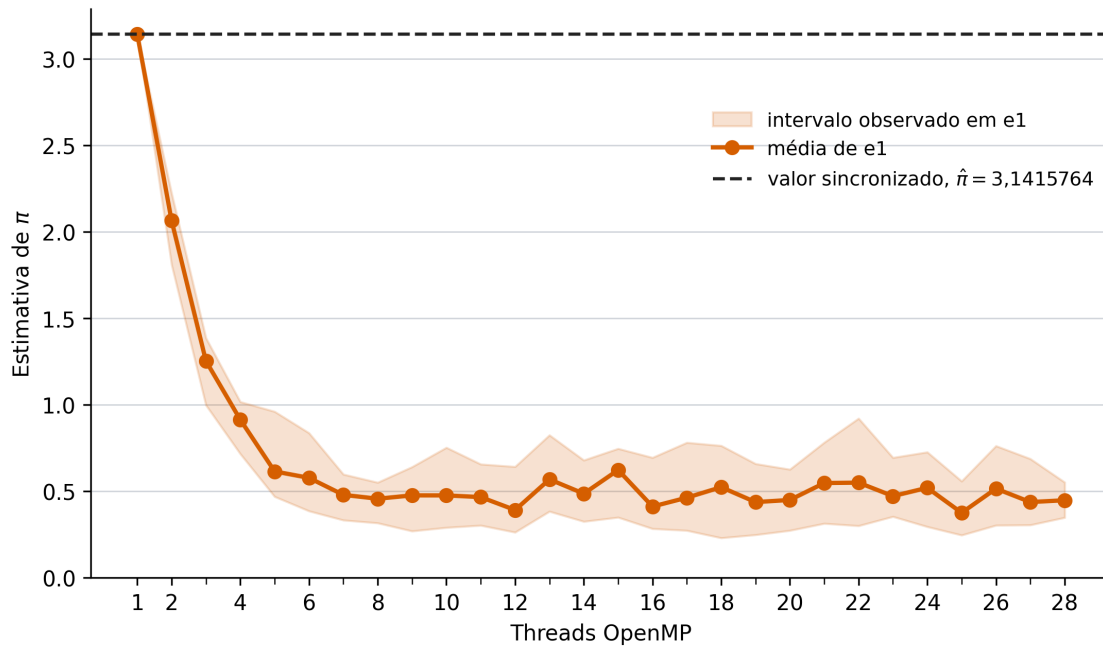


Figure 1: Média e intervalo das dez execuções de e1.c. A linha tracejada indica a estimativa sincronizada com os mesmos pontos.

A Figura 1 mostra que o efeito não segue uma curva regular. A ordem das leituras e escritas muda em cada execução, portanto não há como prever quantos incrementos serão perdidos. O programa compila e termina sem falha. A comparação com uma versão sincronizada é o que revela o defeito.

4.2 Custo da seção crítica

A Tabela 2 compara os tempos de e2.c e e3.c. Em e2.c, o tempo mediano subiu de 0,096032 s com uma thread para 1,221722 s com 28. O trabalho geométrico é paralelo, mas aproximadamente $\pi N/4$, ou 3,93 milhões, de pontos entram no círculo e disputam a seção crítica. Com mais threads, a disputa cresce e o tempo não cai; ele flutua entre 0,9 e 1,5 s desde a oitava thread, com desvios padrão altos.

Table 2: Tempo mediano em segundos. A razão compara as medianas de e2.c e e3.c.

Threads	e2, critical por ponto	e3, contador local	Razão e2/e3
1	0,096032	0,055613	1,73×
2	0,318125	0,027523	11,56×
4	0,570798	0,014342	39,80×
8	0,902719	0,009420	95,83×
14	0,746605	0,003498	213,44×
16	0,902004	0,003321	271,58×
28	1,221722	0,009430	129,56×

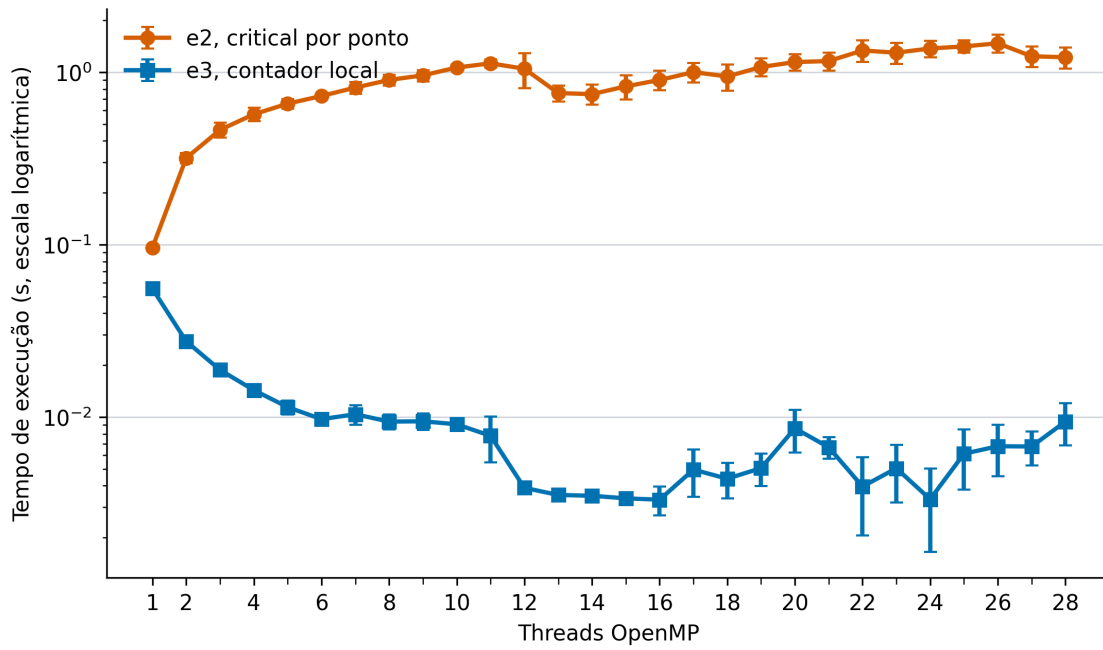


Figure 2: Tempo mediano de execução. As barras de erro mostram o desvio padrão de dez repetições. A escala vertical é logarítmica.

Em e3.c, cada thread entra em `critical` uma vez, depois de processar o seu bloco de iterações. O tempo caiu de 0,055613 s com uma thread para 0,003498 s com 14 threads, um ganho de 15,9×. Com 16 threads, o tempo foi 0,003321 s, o menor valor medido. Nas duas configurações o ganho é próximo de 16, a quantidade de threads lógicas. Com 28 threads, o tempo voltou para 0,009430 s. Nesta máquina, o melhor resultado ocorreu com os 14 núcleos físicos ou logo acima deles.

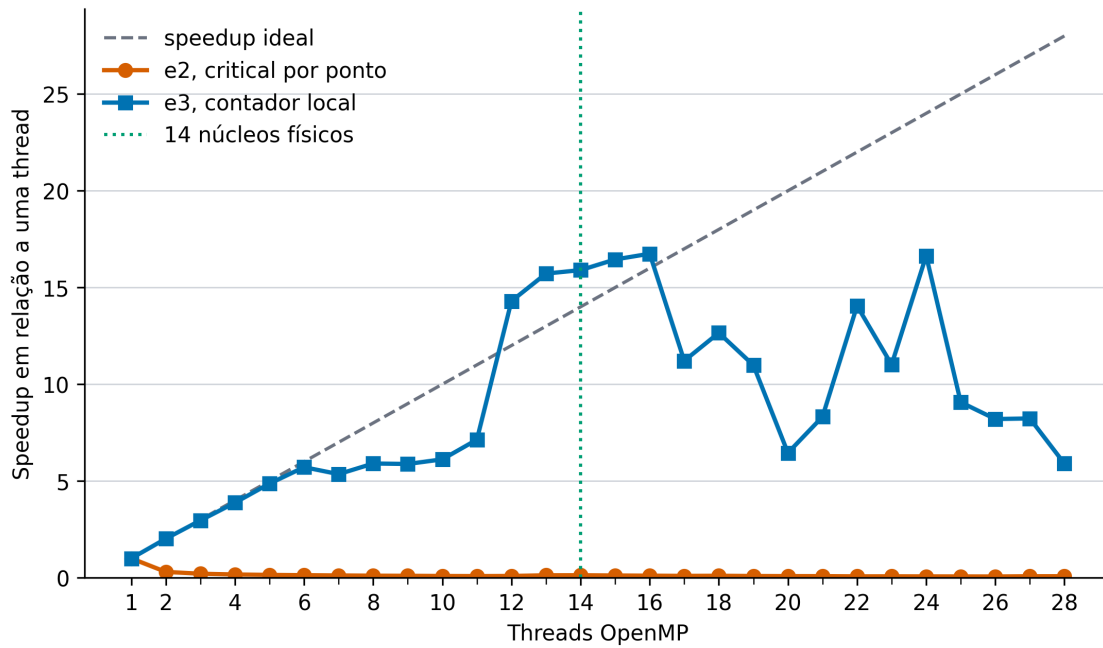


Figure 3: Speedup de e2.c e e3.c em relação a uma thread. A linha verde marca 14 núcleos físicos.

A Figura 3 separa os dois efeitos. O speedup de e2.c fica abaixo de 1 a partir de duas threads, pois a sincronização custa mais do que o trabalho dividido economiza. Em e3.c, o speedup acompanha o número de threads até o entorno de 14 e o excesso de threads lógicas passa a competir pelos mesmos recursos de execução, aumentando o tempo e a variação.

4.3 Comportamento das cláusulas de escopo de dados

Quatro programas complementares foram executados para avaliar o comportamento isolado de cada cláusula de escopo:

- `private(x)` (e4_private.c): cada thread recebe uma instância não inicializada de `x` em sua pilha privada. A leitura de `x` antes de qualquer atribuição resulta em valor indefinido (lixo de memória). Ao final da região paralela, a variável externa preserva o valor original (100).
- `firstprivate(x)` (e4_firstprivate.c): cada thread recebe uma cópia local inicializada com o valor prévio da variável (100). Modificações internas afetam apenas o escopo local de cada thread, e o valor fora da região paralela permanece 100.
- `lastprivate(resultado)` (e4_lastprivate.c): ao término do laço paralelo de 20 iterações ($i \in [0, 19]$), a variável externa recebe deterministicamente o valor computado na última iteração lógica ($19^2 = 361$), independentemente da thread que finalizou por último.
- `default(none)` (e4_default_none.c): ao omitir a classificação da variável `y`, o compilador GCC detecta a variável não declarada e aborta a compilação:

```
1 error: 'y' not specified in enclosing 'parallel'
```

Esse mecanismo estático previne efeitos colaterais e corridas acidentais em programas paralelos complexos.

5 Conclusão

A versão e1.c mostra que um contador compartilhado não pode receber incrementos concorrentes sem sincronização. e2.c preserva a contagem, mas serializa milhões de atualizações e perde desempenho conforme o número de threads aumenta. e3.c acumula localmente e sincroniza apenas no final. Com 14 threads, a versão levou 0,003498 s, contra 0,746605 s de e2.c na mesma configuração. O teste também mostra o limite prático do hardware: depois dos 14 núcleos físicos, as threads lógicas não melhoraram o tempo de e3.c e ainda aumentaram a variância das medições.